WOD2Sim: Contract-Based System Integration of Dataset-Trained Driving Policies into Distributed Closed-Loop Simulation

Alba Maria Tellez Fernandez
Independent Researcher
Email: amtellezfernandez@gmail.com

***Abstract*—Offline driving-policy interfaces hide system failures that appear when the same policy is run as a persistent service in a distributed closed-loop simulator. WOD2Sim treats that transition as a contract-based integration problem rather than as format translation. It defines semantic, temporal, lifecycle, deployment, and evidence contracts for running Waymo Open Dataset (WOD)-style trajectory policies through an AlpaSim external-driver boundary. The reference implementation preserves route geometry, adapts policy horizons to the simulator clock, isolates optional model backends, records launch provenance, and separates claim-valid evidence from diagnostic traces. Current artifacts configure 145 SII evaluation rows across core, ablation, lifecycle, and fault-injection matrices. In the present environment, 55 public synthetic rows execute: 20/20 hardened lifecycle cycles survive, and 15/15 configured faults are detected and localized. The remaining 90/145 rows are closed-loop scene rows blocked before launch, so no claim-valid closed-loop result exists. This outcome prevents missing route proxies, runtime preconditions, or incomplete evidence from being reported as policy performance. The paper contributes the contract taxonomy, implementation boundary, and reproducible evidence scaffold needed to turn blocked integration states into auditable closed-loop experiments.**

I. Introduction

Dataset-trained driving policies and closed-loop simulators expose different failure surfaces. A WOD-style policy consumes logged observations, route intent or route geometry, and a short-horizon ego-relative trajectory target. A closed-loop external driver must instead remain alive as a service, accept incremental sensor and route messages, return controls on the simulator clock, and leave an evidence trail that distinguishes policy behavior from integration failure.

This distinction matters because a wrapper can make integration failures look like policy failures. For example, reducing route geometry to a high-level command can remove the local polyline used by a route-conditioned trajectory policy. A later collision, route drift, or invalid trajectory may then be attributed to policy quality even though the decision-relevant input was lost at the simulator boundary. Similar confounds arise from mismatched trajectory sampling grids, stale observations, duplicate close messages, optional backend imports, missing scene assets, and audit artifacts that cannot support scientific claims.

WOD2Sim addresses this problem as system integration. It provides a reference boundary for running heterogeneous trajectory policies through AlpaSim's external-driver interface while making each boundary assumption inspectable. The central claim is not that WOD2Sim is a new driving policy or an official WOD benchmark implementation. The claim is that dataset-trained policies require explicit semantic, temporal, lifecycle, deployment, and evidence contracts before closed-loop results can be interpreted.

This paper asks two questions: which contract mismatches prevent dataset-trained policies from operating as reliable simulator services, and can explicit contract validation distinguish integration and runtime failures from policy behavior failures? The current repository does not yet support a closed-loop SII result. Instead, it provides a reproducible blocked-results package whose denominators, failure layers, and generated paper assets are traceable to repository artifacts.

TABLE I
Contract map for integrating WOD-style trajectory policies into distributed closed-loop simulation.

Mismatch	Contract	Mechanism	Validation
Command-only route	Semantic	Preserve route geometry	route-source audit
Policy horizon/runtime grid	Temporal	Deterministic resampling	cadence tests
Script flow/session service	Lifecycle	Idempotent late-event handling	lifecycle tests
Implicit host state	Deployment	Materialized manifests	readiness checks
Process exit/evidence	Evidence	Audit-valid summaries	claim gate

II. Integration Problem and Requirements

The offline-policy contract and the external-driver contract differ along five layers. Table I summarizes the recurring mismatches and the WOD2Sim mechanisms used to expose them.

A. Semantic Contract

The semantic contract preserves decision-relevant meaning across the dataset-policy and simulator boundary. Route geometry must reach prediction time as route geometry; a left/straight/right command remains useful context but cannot replace a local polyline. The policy-facing route must follow the ego travel lane rather than an unrelated road center. Ego history, speed, acceleration, camera availability, static hazards, and dynamic actors must retain consistent geometry and motion fields where those fields are available. Visibility diagnostics must not fabricate obstacles, and command-proxy route fallbacks must be labeled diagnostic rather than claim-valid.

B. Temporal Contract

The temporal contract aligns policy outputs with the runtime clock. Source and target horizons are explicit, timestamps are monotonic, and the current ego origin anchors interpolation. Matching grids preserve the trajectory, while mismatched grids are resampled deterministically and headings are recomputed on runtime points. Non-finite, malformed, or structurally invalid trajectories must be rejected or handled safely.

C. Lifecycle Contract

The lifecycle contract turns a policy into a persistent service. Session creation, active state, close, cleanup, and duplicate close are explicit. Late image, egomotion, route, and close events are counted and traced instead of crashing the service. Session state must not leak across repeated or interleaved sessions, and fatal errors must be distinguished from recoverable lifecycle warnings.

D. Deployment and Evidence Contracts

The deployment contract materializes the run: driver and wizard commands, topology, ports, scene identifiers, timeout, policy configuration, external-driver address, simulator Git state, patch state, Docker image identity, GPU state, and checkpoint hashes where applicable. The evidence contract defines when a rollout can support a paper claim: unique run identifiers, immutable manifests, terminal status, route-source and sensor-freshness validity, conformance checks, audits, support-bundle status, failed-run retention, and hashes tying generated numbers to artifacts.

III. WOD2Sim Architecture

Figure 1 shows the system boundary. WOD2Sim sits between an offline or dataset-trained trajectory policy and the distributed AlpaSim runtime. The adapter reconstructs policy-facing state, preserves route geometry, resamples outputs, and isolates model discovery from unrelated optional backends. The runtime side records deployment state and evidence artifacts before any claim is accepted.

The implementation registers dependency-light baselines and learned-policy adapter entry points through AlpaSim model discovery. Constant-velocity and route-following baselines do not require learned checkpoints. Token BC/DAgger and direct actor-aware planning entry points are present, but their execution requires the relevant local checkpoint or actor-proxy artifacts. The current SII matrix therefore treats missing direct-actor proxy state as a deployment blocker, not as a policy failure.

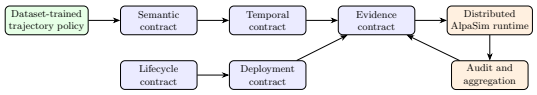

Fig. 1. WOD2Sim localizes integration failures at contract boundaries between a WOD-style policy and a distributed AlpaSim runtime.

IV. Implementation and Evaluation Method

The SII artifact package uses deterministic configuration files for core, semantic ablation, temporal ablation, lifecycle stress, and fault-injection matrices. The core matrix is configured for three policies, six locally cached front-camera scenes, and three seeds. The semantic and temporal ablations each configure paired route or resampling variants on three scenes and three seeds. Lifecycle stress configures repeated service-harness cycles, and the fault matrix enumerates semantic, temporal, lifecycle, plugin, deployment, and evidence injections.

Scene selection is intentionally conservative. Six scenes are selected from available local 26.02 front-camera USDZ artifacts, but the repository does not currently provide authoritative metadata proving straight, intersection, lane-change, dense-traffic, occlusion, or merge categories. The manifest therefore records them as available but unclassified, and the paper does not claim scenario-category coverage.

The evaluation runner expands matrices into rows, writes CSV and JSON summaries, and returns nonzero status when rows are blocked. Aggregation preserves failure rows, rejects duplicate completed run identifiers, writes LaTeX tables with data hashes, and generates vector figures. In the current environment, public synthetic lifecycle and fault-injection harness rows execute, while closed-loop scene rows remain blocked before launch. The resulting rows document the exact experiment denominator while preventing unexecuted closed-loop protocols from becoming results.

Fig. 2. The SII pipeline separates setup, readiness, manifest materialization, rollout, audit, aggregation, and paper-asset generation.

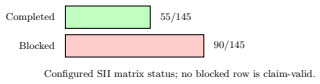

Fig. 3. Configured matrix status. The denominator is 145 configured rows; 55 rows completed and 90 rows are blocked.

TABLE II
CURRENT SII MATRIX STATUS. ROWS ARE CONFIGURED EVIDENCE ROWS, NOT COMPLETED CLOSED-LOOP ROLLOUTS.

Status	Rows	Attempted	Completed
All configured	145	55	55
Blocked	90	0	0

V. RESULTS

No SII closed-loop result is claim-valid in the current repository state. Table II reports the aggregate configured status generated from ‘artifacts/sii2027/results/summary.json‘ and ‘runs.csv‘.

The configured matrices contain 145 rows: 54 core rows, 18 semantic-ablation rows, 18 temporal-ablation rows, 40 lifecycle rows, and 15 fault-injection rows. The current aggregate attempts 55 public synthetic rows, completes 55 rows, and marks 90 closed-loop rows as blocked. Consequently, route progress, collision rate, off-road rate, closed-loop service-crash rate, latency percentiles, and real-scene ablation effects are unavailable for SII claims.

TABLE III
PUBLIC SYNTHETIC DIAGNOSTICS. THESE ROWS EXERCISE SERVICE-LEVEL HARNESSES ONLY; THEY ARE NOT CLOSED-LOOP SCENE ROLLOUTS.

Synthetic diagnostic	Count	Total
Lifecycle hardening survived	20	20
Pre-hardening survived	0	20
Faults detected	15	15
Faults localized	15	15

The public synthetic diagnostics in Table III show that the hardened lifecycle harness survives 20/20 cycles, while the strict/pre-hardening comparison survives 0/20 cycles. The fault harness detects 15/15 configured injections and localizes 15/15 to the expected contract layer/code. These are diagnostic harness results, not policy-performance or real-scene reliability metrics.

The most important result is therefore diagnostic rather than behavioral: the evidence contract blocks all closed-loop scene rows from being reported as policy performance. This is the

desired behavior for a system-integration paper. A missing actor proxy, unexecuted matrix, or absent scene-category evidence is surfaced as an integration or evidence condition rather than hidden inside a policy metric.

VI. Discussion, Limitations, and Related Work

The framework generalizes beyond a single simulator because the contracts identify boundary assumptions rather than AlpaSim-specific filenames. Route geometry, sampling cadence, lifecycle robustness, deployment provenance, and claim-valid evidence also matter in WOD-style datasets [1] and in closed-loop planners and simulators such as nuPlan [2], CARLA [3], Waymax [4], and NAVSIM [5]. WOD2Sim uses AlpaSim as the case study because its external-driver boundary makes these contracts explicit.

The limitations are material. WOD2Sim is not an official Waymo challenge interface, does not reconstruct complete WOD worlds, and does not redistribute restricted scene assets. The current SII package has no executed closed-loop matrix, no validated scene-category coverage, and no direct-actor proxy for direct actor-aware planning. Its lifecycle and fault-injection evidence comes from public synthetic harnesses, not simulator-backed scene rollouts. Learned token policies remain outside the public baseline unless a legitimate local checkpoint and hash are provided.

These limitations also explain why the paper does not compare policy quality. A framework paper can report reliability, validity, and diagnostic coverage only after the corresponding rows are executed and audited. Until then, the strongest supported claim is that the repository materializes and blocks incomplete evidence rather than allowing it to appear as a benchmark.

VII. Conclusion

Integrating a dataset-trained trajectory policy into distributed closed-loop simulation requires more than input/output conversion. WOD2Sim formulates the boundary as five contracts: semantic, temporal, lifecycle, deployment, and evidence. The current repository implements the contract-oriented release surface and a reproducible SII artifact scaffold, but the generated SII aggregate reports no claim-valid closed-loop execution. That blocked status is a useful integration outcome: it keeps runtime, precondition, and evidence failures separate from policy behavior until real audited rollouts exist.

References

[1] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2446–2454, 2020.

[2] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari, "nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles," in *CVPR Workshop on Autonomous Driving*, 2021.

[3] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 1–16, PMLR, 2017.

[4] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mougin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp, "Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research," in *Advances in Neural Information Processing Systems*, 2023.

[5] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems*, 2024.